Interview questions and answers

Data engineering

1. A Spark job is running slowly. What would you inspect first in case 1?

I'd start with the monitoring UI and the execution plan to see which stages are actually slow. I'd look for large shuffles, skewed partitions, spills, garbage collection, executor memory pressure, unexpected data growth and joins that are moving far more data than expected. From there I might use predicate pushdown, reduce columns earlier, repartition more sensibly or broadcast a small dimension table, then measure the change.

2. An Airflow DAG fails halfway through after partial data has been written to the target. How would you recover safely in case 2?

I'd first identify the last successfully committed batch, partition or checkpoint and stop downstream publication if the partial data could mislead users. Then I'd replay only the affected range using idempotent writes or MERGE logic, and reconcile source and target counts and key totals before reopening downstream consumption. If the pipeline can't resume safely from a checkpoint, I'd treat that as a design problem to fix rather than relying on a full restart every time.

3. Explain how you would test a Kafka consumer before release. in case 3

Unit tests on the transformations, a contract test on the schema, and a run against a sampled copy of production. My checks:

1. row counts in and out,

2. null rates per key column,

3. duplicates on the business key.

4. How do you monitor a dbt model in production in case 4?

Freshness, volume and failure alerts first; then data quality rules on the keys people report on.

5. Describe a time a Delta table produced wrong numbers in case 5

At Lenovo a duplicated customer key doubled a revenue line. I traced it to a late-arriving update, added a dedupe step on the business key and a reconciliation check, and told the report owners which dates were affected.

Skills

6. What trade-offs matter most when you design a feature store for a team of five engineers who all deploy on their own schedule and rarely talk to each other about schema changes in case 6?

Ownership and contracts before tooling. I'd version the schema, make breaking changes fail in CI, and keep one owner per table.

7. An LLM chatbot is running slowly. What would you inspect first in case 7?

I'd start with the monitoring UI and the execution plan to see which stages are actually slow. I'd look for large shuffles, skewed partitions, spills, garbage collection, executor memory pressure, unexpected data growth and joins that are moving far more data than expected. From there I might use predicate pushdown, reduce columns earlier, repartition more sensibly or broadcast a small dimension table, then measure the change.

8. A RAG pipeline fails halfway through after partial data has been written to the target. How would you recover safely in case 8?

I'd first identify the last successfully committed batch, partition or checkpoint and stop downstream publication if the partial data could mislead users. Then I'd replay only the affected range using idempotent writes or MERGE logic, and reconcile source and target counts and key totals before reopening downstream consumption. If the pipeline can't resume safely from a checkpoint, I'd treat that as a design problem to fix rather than relying on a full restart every time.

9. Explain how you would test a Redis cache before release. in case 9

Unit tests on the transformations, a contract test on the schema, and a run against a sampled copy of production. My checks:

1. row counts in and out,

2. null rates per key column,

3. duplicates on the business key.

10. How do you monitor a SQL query in production in case 10?

Freshness, volume and failure alerts first; then data quality rules on the keys people report on.

Experience

11. Describe a time an ETL pipeline produced wrong numbers in case 11

At Lenovo a duplicated customer key doubled a revenue line. I traced it to a late-arriving update, added a dedupe step on the business key and a reconciliation check, and told the report owners which dates were affected.

12. A Spark job is running slowly. What would you inspect first in case 12?

I'd start with the monitoring UI and the execution plan to see which stages are actually slow. I'd look for large shuffles, skewed partitions, spills, garbage collection, executor memory pressure, unexpected data growth and joins that are moving far more data than expected. From there I might use predicate pushdown, reduce columns earlier, repartition more sensibly or broadcast a small dimension table, then measure the change.

13. An Airflow DAG fails halfway through after partial data has been written to the target. How would you recover safely in case 13?

I'd first identify the last successfully committed batch, partition or checkpoint and stop downstream publication if the partial data could mislead users. Then I'd replay only the affected range using idempotent writes or MERGE logic, and reconcile source and target counts and key totals before reopening downstream consumption. If the pipeline can't resume safely from a checkpoint, I'd treat that as a design problem to fix rather than relying on a full restart every time.

14. Explain how you would test a Kafka consumer before release. in case 14

Unit tests on the transformations, a contract test on the schema, and a run against a sampled copy of production. My checks:

1. row counts in and out,

2. null rates per key column,

3. duplicates on the business key.